Search 搜索

Informed Search Methods 启发式搜索/有信息搜索

UCS 要探索所有节点=>bad，浪费很多时间和空间

-Heuristics 启发式算法

A heuristic is：a function that estimates how close a state is to a goal / Designed for a particular search problem (在实际计算代价之前，先进行预测，以此不必检测所有 nodes，节约时间和空间。那么需要设计一个用于预测的启发式函数) / Examples：Manhatten distance(曼哈顿距离，即上下左右四个方向情况下的起点与终点横竖距离绝对值的相加) Euclidean distance (欧几里得距离即直线距离) for pathing

Greedy Best-First Search 贪婪最佳优先搜索

Strategy: expand a node that you think is closest to a goal state （并不是实际上最近的，而是通过你的启发式算法 Heuristics 算出来的你所认为最近的) (每一步选择时并不会考虑这一步之前的代价，只会考虑在 DFS 的 fringe 即边缘里面选择代价最小的一个 h(n))。

贪心搜索可以快速找到最优解，但可能直接得到解/错解。

不具备完备性和最优性

最坏情况下的时间和空间复杂度为 O (b^m)

A*Search

Combining UCS and Greedy <= **Uniform-cost** orders by path cost, or backward cost **g(n)** 已经消耗的代价(真实的代价) / **Greedy** orders by goal proximity, or forward cost **h(n)** 预测的从当前位置到目标所需的代价(非真实代价)

=>A* Search orders by the sum: **f(n) = g(n) + h(n)**

不是遇到 goal node 之后就停下，而是找到最小代价的到达 goal node 的路径后才停下。

最优性：If the heuristic is admissible, the tree-search version of A* is optimal

A heuristic h is admissible (optimistic) if: $0 \le h(n) \le h^*(n)$

启发式函数计算的代价要保证不超过真实代价

If the heuristic is consistent, the graph-search version A* is optimal(比 admissible 要求更高，consistent=>admissible。树搜索可以遍历每条路线，但是若有两条路线 sacg 和 sbcg，图搜索在选择遍历 sacg 路径后已经遍历过 c 结点，因此遍历 sbc 想扩展 c 结点时无法再次在此处扩展遍历(c 会被 skip)，这会导致找到的路线必定是搜索的第一条)

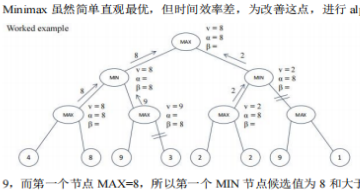
Consistency: heuristic "arc" cost ≤ actual cost for each arc $h(A) - h(C) \le cost(A \text{ to } C)$不仅需要到终点的启发式函数预估代价小于实际代价，还需要每一步预估代价都小于实际代价。

A*具有完备性！A*对于任何一致性启发式算法都最优，无其他最优算法保证扩展更少节点。

Alpha-beta Pruning 剪枝　敌对搜索

Minimax 虽然简单直观最优，但时间效率差，为改善这点，进行 alpha-beta 剪枝。

Worked example



在这个例子中，当访问到第二个 MAX 第一个子节点值为 9 时，无论后面的子节点如何取值，这个最大值节点一定大于等于 9，而第一个节点 MAX=8，所以第一个 MIN 节点候选值为 8 和大于等于 9，只能取 8，因此我们不再需要看第二个节点的第二个子节点了。同理，第三个 MAX=2，因此第二个 MIN 节点一定小于等于 2，而 MAX 根节点的候选值为 8 和小于等于 2，则无论最右 MAX 取多少，根节点都一定取 8。这正是为什么我们可以直接抛弃子节点，从而对这棵搜索树进行剪枝。这可能致中间值错误，但结果始终正确。

剪枝的有效性 efficiency 高度依赖于检查状态的顺序；在完美的情况下，alpha-beta 只需要检查 O(b^(m/2))个节点(时间复杂度)；随机情况下，也只有 O(b^(3m/4))的时间复杂度

图例中，倒三角是 MIN，正三角是 MAX。

Constraint Satisfaction Problems 约束满足问题

A set of variables X={X1,....,Xn} 变量

A set of domains D={D1,....,Dn} 域

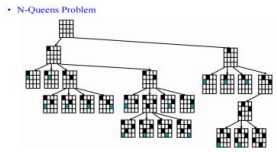
A set of constraints C 约束条件

状态是通过将域的值赋值给部分或全部变量来定义的。

Backtracking Search 回溯搜索

回溯搜索是求解 CSP 的基本 uninformed 算法(无信息)。

回溯= DFS +变量排序+违例失败　Backtracking search is the depth-first search with two improvements:1.One variable at a time 一次一个变量 2.Backtrack when a variable has no legal values 当变量没有合法值时回溯。左是一个例子。每次针对一个变量，无法继续拓展且未找到解就回溯到选择的地方。

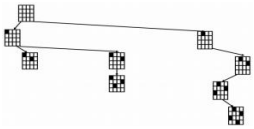


Inference in CSPs：

Inference 推理:约束传播，减少一个变量的合法值的数量。约束传播可以与搜索交织在一起，或者作为一个预处理步骤。关键思想是局部一致性 local consistency。

1.Forward checking 前向检查

前向检查：划掉添加到现有赋值时违反约束的值；将信息从已赋值变量传播到未赋值变量；跟踪未分配变量的域，当任何变量没有合法值时终止搜索。如右上，当 Q1 取在(1, 1)时，就知道标蓝色的地方之后都不能取(Qn 表示第 n 行取值)。其他仍按照回溯搜索的规则，最终得到下图，这就是使用前向检查时得到结果。

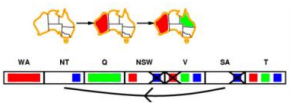


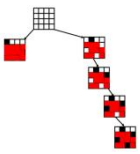
2.Arc Consistency 弧相容

An arc X -> Y is consistent if for every x there is some y which could be assigned without violating a constraint 对于每个 X 都有一个 Y 可以在不违反约束的情况下被赋值。

前向检查:强制指向每个新赋值的弧的一致性。

在确定 Arc Consistency 的时候只在乎这条 arc，并不在乎整图是否能成立。牢记对 every x in the tail,都有 some y in the head！注意在删去操作之后，需要重新对之前做过涉及该删去点的 arc 进行 recheck 重新检查，因为之前的 arc 可能会因为该删去操作变成(in)consistent。如果在一次删去操作后，有一个地点没有可赋值了，那么终止。弧相容比前向检查更早检测到故障。可作为预处理器也可在每次赋值后运行。





左图是弧相容的 n 皇后结果。其第一步和前向检查第一步一样，但第二步是在第一步基础上继续检查，直到发现故障/结果。

Arc-Consistency Algorithm (AC-3)：应用 AC-3 后:要么每个弧都是弧一致的，要么某个变量有一个空域，说明 CSP 无法求解。假设一个 CSP 有 n 个变量，d 个域大小和 c 个二进制约束；每个弧(Xk, Xi)可插入 d 次；检查弧的一致性需要 O(d^2)时间,所以总的最坏情况时间是 O(cd^3)。

局限性：运用弧一致后，可能有多个/一个/没有解决方案。可以帮助更快接近答案/排除错误，但仍要和回溯搜索一起用，无法仅凭弧一致解决 CSP 问题。

K-consistency K 一致性

1-Consistency (Node Consistency): 每个单个节点的域有一个满足该节点一元约束的值

/2-Consistency(弧一致性):对于每一对节点，任何对一个节点的一致分配都可以扩展到另一个节点/ k- consistency:对于每 k 个节点，任何对 k-1 的一致分配都可以扩展到 kth 节点。k 值越大，计算代价越大。

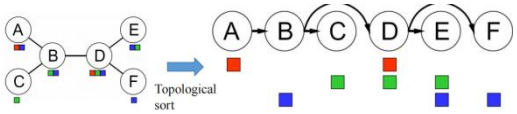
Strong n-consistency means we can solve without backtracking

但工作量非常大，不如用前向检查等来进行一些 tradeoffs

3.The Structure of Problems-Tree-Structured CSPs

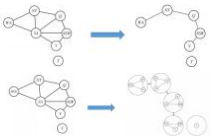
也可以通过改变问题结构来简化问题，把图结构改成树结构

首先，在 CSP 的约束图中任选一个节点来作为树的根节点 将树中的所有无向边转换为指向根节点反方向的有向边。然后将得到的有向无环图线性化（linearize）(或拓扑排序（topologically sort））。简单来说，这也就意味着将图的节点排序，让所有边都指向右侧。



执行弧一致性的后向传递（backwards pass）。从 i = n 迭代至 i = 2，要求所有的弧 Parent(Xi),Xi 都满足弧相容。最后，执行前向赋值（forward assignment）。从 X1 到 Xn，给每个 Xi 赋一个与其父亲一致的值。

Nearly Tree-Structured CSPs 近似树形结构



将一般约束图简化为树：1. 删除节点(切割集条件:实例化(以各种方式)一组变量，例如剩下的约束图是一棵树；割集大小 c 给出运行时间 $O(d^c (n-c) d^2)$，对于小 c 来说非常快)　2.一起折叠节点(树分解:将约束图构造成一组相互连接的子问题)

Knowledge and Reasoning 知识与推理

Knowledge-based Agent 基于知识的 Agent

基于知识的智能体由知识库 knowledge base(知识表示语言中的一组句子，由其语法和语义定义。1.命题逻辑 2.一阶逻辑)和推理机制 inference mechanism(从旧句子中推出新的句子)组成。 例：知识：A->B,B->C,通过推理：A->C

Propositional Logic 命题逻辑

以下符号按顺序为非(Not),与(And)/ 合取(Conjuction),或(Or)/ 析取(Disjunction),蕴涵(Implication),双条件(Biconditional)

$\neg P$	is true iff	P	is false		
$P_1 \wedge P_2$	is true iff	P_1	is true **and**	P_2	is true
$P_1 \vee P_2$	is true iff	P_1	is true **or**	P_2	is true
$P_1 \Longrightarrow P_2$	is true iff	P_1	is false **or**	P_2	is true
i.e.,	is false iff	P_1	is true **and**	P_2	is false
$P_1 \Leftrightarrow P_2$	is true iff	$P_1 \Longrightarrow P_2$	is true **and**	$P_2 \Longrightarrow P_1$	is true

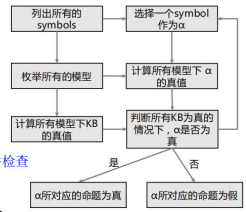
P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

$P \Longrightarrow Q$　*Equal to*　$\neg P \vee Q$

以上为构建知识库方法，由其推理得真值表，计算 KB 真值。

➢ **TT-Entails 算法**

1. 列出所有的symbols；
2. 枚举所有可能的模型；
3. 对于每个模型，计算KB的真值；
4. 选择一个α（通常为其中一个symbol或者说是新的命题），如果在每个 KB 为真的模型中，α也为真，则KB ⊨ α，表示α所对应的命题为真。



这里的 4 就是 model checking：列举模型并检查

α在每个 KB 为真的模型中是否为真。

Inference 推论 logical equivalence 语义等价

- Two sentences are logically equivalent iff true in same models: $\alpha \equiv \beta$ if and only if $\alpha \vDash \beta$ and $\beta \vDash \alpha$

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Longrightarrow \beta) &\equiv (\neg\beta \Longrightarrow \neg\alpha) && \text{contraposition} \\
\boxed{(\alpha \Longrightarrow \beta) \equiv (\neg\alpha \vee \beta)} && && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Longrightarrow \beta) \wedge (\beta \Longrightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

真值指派：如果一个 KB 有 K 个原子命题，P1 到 Pk,我们设置 true 或者是 false，所有的复杂句子都是有这些原子命题运算构成的，则一共有 2^k 种真值指派。如果 in all models 这个 sentence 都为 true=>is valid,如果 sentence is true in no models=>is unsatisifiable.如果 sentence is true in some models=>is satisfiable

$KB \vDash \alpha$ if and only if $(KB \wedge \neg\alpha)$ is unsatisfiable

Inference：Proof by Resolution 归结原理

Conjunctive Normal Form CNF 合取范式：conjunctions of disjunctions of literals/clauses(一堆析取的合取)。一个原子命题的符号 A 或者¬A 都称为 literals 文字。

归结原理的第一步是把 KB(knowledge base)改写成 CNF 形式(通过语义等价)，即只有与、或和非，且子式间只为与。

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate ⇔, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Longrightarrow \beta) \wedge (\beta \Longrightarrow \alpha)$.
$$(B_{1,1} \Longrightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Longrightarrow B_{1,1})$$
2. $\boxed{\text{Eliminate } \Rightarrow, \text{ replacing } \alpha \Rightarrow \beta \text{ with } \neg\alpha \vee \beta.}$
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$
3. Move ¬ inwards using de Morgan's rules and double-negation:
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$
4. Apply distributivity law (∨ over ∧) and flatten:
$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

有了 CNF 之后，想要证明 KB |=a，使用 unsatisfied 前式。即想要证明结论，可以先假设结论是错的，导致矛盾，没有任何一个世界，可以 satisfiable 这样一个 sentence（KB ^ ¬ a）。若最后出现空子句，则可以证明原命题为真。(最右出现)若直到无法产生新 clause 时都未产生空子句，则原命题为假。

Resolution Rule:

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \qquad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where ℓ_i and m_j are complementary literals. E.g.,

$$\frac{P_{1,3} \vee P_{2,2}, \qquad \neg P_{2,2}}{P_{1,3}}$$

可看到 CNF 中析取式都在上排，下排是归结定理给出的新 clause 子句。归结定理如左：

归结定理上 2 式推下 1 式。

所以上例最后如右：

先改写成 CNF，然后写出观察到的因素和要证明的结果，再利用归结定理证明。

Resolution is sound 可靠性 and complete 完备性 for propositional logic.

- $KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}))$ reformulated as: $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$
- Observation: $\neg B_{1,1}$
- Goal: disprove: $\alpha = \neg P_{1,2}$
- Resolution

$$\frac{\neg P_{1,2} \vee B_{1,1} \quad \neg B_{1,1}}{\neg P_{1,2}}$$

- Resolution

$$\frac{\neg P_{1,2} \quad P_{1,2}}{false}$$

Probabilistic Reasoning 概率推理

Joint Distributions 联合分布

- A joint distribution over a set of random variables: $X_1, X_2, ..., X_n$ specifies a real number for each assignment (or outcome):

$$P(X_1 = x_1, X_2 = x_2, ..., X_N = x_N)$$
$$P(x_1, x_2, ..., x_N)$$

- Must obey: $P(x_1, x_2, \dots x_n) \geq 0$

$$\sum_{(x_1, x_2, \dots x_n)} P(x_1, x_2, \dots x_n) = 1$$

Marginal Distributions 边际分布

$P(T, W)$

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

$$P(t) = \sum_w P(t, w) \longrightarrow P(T)$$

T	P
hot	0.5
cold	0.5

$$P(w) = \sum_t P(t, w) \longrightarrow P(W)$$

W	P
sun	0.6
rain	0.4

$$P(X_1 = x_1) = \sum_{x_2} P(X_1 = x_1, X_2 = x_2)$$

hidden (unobserved) variables

Conditional Distributions 条件分布

$$P(x_1|x_2) = \frac{P(x_1, x_2)}{P(x_2)} = \frac{P(x_1, x_2)}{\sum_{x_1} P(x_1, x_2)}$$

Bayes Rule 贝叶斯法则

$$P(x|y) = \frac{P(y|x)}{P(y)} P(x)$$

$$P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause}) P(\text{cause})}{P(\text{effect})}$$

Conditional independence 条件独立

X is conditionally independent of Y given Z $X \perp\!\!\!\perp Y | Z$ if and only if:

$$\forall x, y, z : P(x, y|z) = P(x|z) P(y|z)$$

or, equivalently, if and only if

$$\forall x, y, z : P(x|z, y) = P(x|z)$$

Independence 独立性 $X \perp\!\!\!\perp Y$

$$\forall x, y : P(x, y) = P(x) P(y)$$

Another form:

$$\forall x, y : P(x|y) = P(x)$$

Chain Rule 链式法则

$$P(x_1, x_2, x_3) = P(x_1) P(x_2|x_1) P(x_3|x_1, x_2)$$
$$P(x_1, x_2, \dots x_n) = \prod_i P(x_i | x_1 \dots x_{i-1})$$

Product Rule 乘积法则

$$P(y) P(x|y) = P(x, y) \longleftrightarrow P(x|y) = \frac{P(x, y)}{P(y)}$$

Bayesian Network 贝叶斯网络

- Inference by enumeration: **Enumeration 枚举**
  - Step 1: Select the entries consistent with the evidence
  - Step 2: Sum out H to get joint of Query and evidence

$$P(Q, e_1 \dots e_k) = \sum_{h_1 \dots h_r} \underbrace{P(Q, h_1 \dots h_r, e_1 \dots e_k)}_{X_1, X_2, \dots X_n}$$

  - Step 3: Normalization

$$P(Q|e_1 \cdots e_k) = \frac{1}{Z} P(Q, e_1 \cdots e_k) \qquad Z = \sum_q P(Q, e_1 \cdots e_k)$$

Join up the whole joint distribution before you sum out the hidden variables/ 计算复杂度 O(d^n)，非常慢！

Variable Elimination 变量消除

Idea: interleave joining and marginalizing!

: it usually much faster than inference by enumeration

举例：

- by Enumeration

$$P(L) = \sum_t \sum_r P(L|t) P(r) P(t|r)$$

Join on r

Join on t

Eliminate r

Eliminate t

- Variable Elimination

$$= \sum_t P(L|t) \sum_r P(r) P(t|r)$$

Join on r

Eliminate r

Join on t

Eliminate t

$$P(\neg B|J, M) = \frac{P(\neg B, J, M)}{P(J, M)}$$
$$= \alpha \sum_A \sum_E P(\neg B, J, M, A, E) \ (\alpha \text{为归一化系数})$$
$$= \alpha \sum_A \sum_E P(\neg B) P(E) P(A|\neg B, E) P(J|A) P(M|A)$$
$$= \alpha P(\neg B) \sum_E P(E) \sum_A P(A|\neg B, E) P(J|A) P(M|A)$$

同理可求得 $P(B|J, M) = 0.0005924\alpha$，故 $P(\neg B|J, M)$ 的归一化概率为

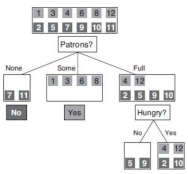
$$\frac{0.00149\alpha}{0.00149\alpha + 0.0005924\alpha} = 71.6\%$$

习算法输出的原因。下面标是 Input Attributes

Decision Trees

Example	Inpu				Goal
	Pat	Type	Est		Wil/Wait
x_1	Some	French	0–10		$y_1 = Yes$
x_2	Full	Thai	30–60		$y_2 = No$
x_3	Some	Burger	0–10		$y_3 = Yes$
x_4	Full	Thai	10–30		$y_4 = Yes$
x_5	Full	French	>60		$y_5 = No$
x_6	Some	Italian	0–10		$y_6 = Yes$
x_7	None	Burger	0–10		$y_7 = No$
x_8	Some	Thai	0–10		$y_8 = Yes$
x_9	Full	Burger	>60		$y_9 = No$
x_{10}	Full	Italian	10–30		$y_{10} = No$
x_{11}	None	Thai	0–10		$y_{11} = No$
x_{12}	Full	Burger	30–60		$y_{12} = Yes$

$$Gain(A) = B(\frac{p}{p+n}) - Remainder(A)$$
$$B(q) = -(q \log_2 q + (1-q) \log_2 (1-q))$$
$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k + n_k}{p+n} B(\frac{p_k}{p_k + n_k})$$

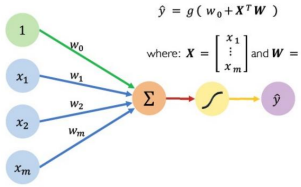


$$Gain(Type) = 1 - [\frac{2}{12} B(\frac{1}{2}) + \frac{2}{12} B(\frac{1}{2}) + \frac{4}{12} B(\frac{2}{4}) + \frac{4}{12} B(\frac{2}{4})] = 0 \text{ bits}$$

$$Gain(Patrons) = 1 - [\frac{2}{12} B(\frac{0}{2}) + \frac{4}{12} B(\frac{4}{4}) + \frac{6}{12} B(\frac{2}{6})] \approx 0.541 \text{ bits}$$

Perceptron 感知器

$$\hat{y} = g(w_0 + X^T W)$$



where: $X = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$ and $W = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$

Output

Linear combination of inputs

$$\hat{y} = g\left(w_0 + \sum_{i=1}^{m} x_i w_i\right)$$

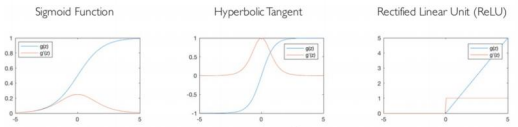
Non-linear activation function

Bias

Inputs Weights Sum Non-Linearity Output

权重 w 更新方法：如果算出ŷ与题目给定 y 相同，correct，不改变；如果预测错了，降低错误答案的对应权重 w*=w*-f(x)，增加正确答案的对应权重 w=w+f(x)；非正确/错误答案的项的权重保持不变。

Activation function 激活函数：全都是非线性的=>在网络中引入非线性，否则最终结果还是现行的。以下列举三种：

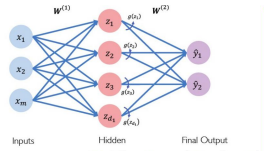


$$g(z) = \frac{1}{1 + e^{-z}} \qquad g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \qquad g(z) = \max(0, z)$$

$$g'(z) = g(z)(1 - g(z)) \qquad g'(z) = 1 - g(z)^2 \qquad g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

Artificial Neural Network ANN 人工神经网络

Single Layer Neural Network 单层神经网络：

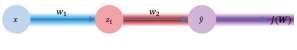


$$z_1 = w_0^{(1)} + \sum_{j=1}^{m} x_j w_{j1}^{(1)} \qquad \hat{y}_1 = g\left(w_0^{(2)} + \sum_{j=1}^{d_1} z_j w_{j1}^{(2)}\right)$$

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

Loss Optimization 损失优化 – Gradient Descent 梯度下降：初始化->循环 3.4 直到收敛



Computing Gradients:(左图)

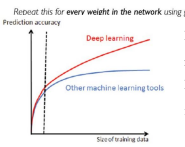
Backpropagation 计算梯度：反向传播

$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

ANN：堆叠感知器形成神经网络；通过反向传播进行优化

Repeat this for every weight in the network using gradients from later layers



Deep Learning 深度学习

Machine Learning with small data：Overfitting 过拟合，reduce model complexity

Machine Learning with big data：Underfitting 欠拟合，increasing model complexity

Conventional Neural Networks CNN 卷积神经网络 下面写了四层

卷积层：使用卷积核进行特征提取和特征映射

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

- The convolutional layer

Examples time:

Input volume: 32x32x3
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10

Number of parameters in this layer?
each filter has 5*5*3 + 1 = 76 params (+1 for bias)
=> 76*10 = 760

权重数+偏置数=层中参数数量

池化层：对特征图进行降采样，从而减少参数量。

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- Note that it is not common to use zero-padding for Pooling layers

降维/保持空间方差/对每个激活映射独立操作

最大池化 max pool 输出:区域中观察到的最大输入值

均值池化 avg pool 输出:区域中观察到的平均输入值

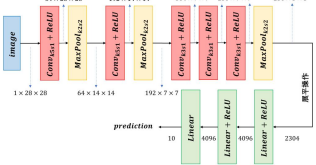
两者最大区别在于卷积核的不同（池化是一种特殊的卷积过程）

激活函数层：引入非线性以逼近更复杂的函数，如 ReLU。

全连接层：通过全连接操作，整合前面提取的特征，实现最终的分类任务。

卷积层参数个数计算方法:(卷积核高*卷积核宽*通道数 +1)*卷积核个数

当前全连接层参数个数计算方法:(上一层神经元个数+1)*当前层神经元个数



以上的 1 代表偏置，因为每个神经元都有一个偏置。

AlexNet：AlexNet 分为卷积和全连接两部分。它的卷积部分由交替的卷积层(Convolution)和最大池化层(Max Pooling)组成，全连接部分为 3 层全连接层。仅使用全连接层无法实现。

简化后的模型如图所示：中间部分为模型的逐层解析，箭头部分指出了经过对应层作用后特征图或特征的尺寸（第一个数表示通道数或特征长度），我们用这些尺寸的变化来表示对应的卷积层或全连接层的输入输出的通道数或特征长度。注意到卷积层和池化层中存在下标，如 **Conv k5s1**，下标中的 **k5** 表示卷积核的大小为 5×5，**s1** 表示卷积核在特征图上滑动时的步长为 **1**。为了模型表示的简洁性，我们用"+"对一些简单的操作进行连接，合并到同一层表示，其对数据的操作采用从左到右的顺序。如 **Conv k5s1+ReLU** 表示先通过一个卷积层，然后对特征作用 ReLU 激活函数。

CNN 为什么适合图像分类任务？

1. 局部感知：卷积层通过局部感受野可以更好地捕捉图像中的局部特征。
2. 参数共享：通过卷积使得相同的参数在不同位置共享，这极大减少了模型的参数量。
3. 平移不变性：卷积能够保持一定的平移不变性。

Markov Decision Processes MDP 马尔可夫决策过程

Non-Deterministic Search 不确定搜索

Agent 所在的环境会迫使 agent 的行为变得不确定(nondeterministic)，意味着在某一状态下采取的一个行动会有多个可能的后继状态。(如采取 Up 动作导致到上一格概率为 0.8，但也有 0.1 的概率到左右两格)。

不确定搜索问题可以用马尔科夫决策过程MDP，"马尔可夫"意味着操作结果仅依赖于当前状态。

Markov Decision Processes

A sequential decision problem with a fully observable environment with a *Markovian transition model* and *additive rewards* is modeled by a Markov Decision Process (MDP)

MDP 的转换函数是一个概率函数，用来表示一个 agent 从状态 s∈S 采取行动 a∈A 后得到状态 s'∈S 的概率。这种转变是 Markovian 马尔可夫的

$$P(s_{t+1} | a, s_0, \dots, s_t) = P(s_{t+1} | a, s_t)$$

An MDP has the following components:

1. A (finite) set of states S
2. A (finite) set of actions A
3. Transition Model: $T(s, a, s') = P(s' | a, s)$
4. Reward Function: $R(s)$
   Sometimes a reward function $R(s, a, s')$

- MDPs are non-deterministic search problems

or now, we'll define the utility 效用 of a sequence of states as the sum of the rewards received.

A sequential decision problem for a fully observable, stochastic environment with a Markovian transition model and additive rewards is called a Markov Decision Process.

Policy: mapping from a state to an action 策略:从状态到动作的映射

需要为所有状态定义，以便 agent 始终知道该做什么

Notation 符号:-π denotes a policy π表示策略

-π(s)denotes the action recommended by the **policy π** for **state s**

-π(s)表示策略π对状态 s 推荐的动作

Many different policies exist for an MDP. Some are better than others. The "best" one is called the optimal policy π*

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s, a, s') U(s')$$

执行策略度量 expected utility 预期效用，即策略生成的可能状态序列的平均效用。

如何计算这个最优策略?不能评估所有的政策，太多了。相反，我们计算 **utility of the states** 状态的效用，然后使用状态效用在每个状态中选择一个最佳动作。

The Bellmen Equation 贝尔曼方程

$$U(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U(s')$$

More generally, we have:

$$U_{i+1}(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U_i(s')$$

左是折扣奖励的最大可能预期总和。如果智能体处于状态 s 并存活 i+1 个时间步，这个方程被称为 Bellman Update。

The Value Iteration 值迭代

值迭代(Value iteration)是一种动态

policy often converges long before the values

$$U_1(s) = R(s)$$
$$U_{i+1}(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U_i(s')$$

值迭代是围绕状态效用的思想设计的。计算难度来自于 bellman 方程的最大运算。与计算状态的一般效用(假设以最优方式行动)不同，更容易计算的是假设一个策略时的状态效用。

1. Apply bellman update until it the utility function converges (to U*(s)).
2. The optimal policy is given by:

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s, a, s') U^*(s')$$

Policy Iteration 策略迭代

结合策略评估和策略提升。无极大算子，所以方程线性，可以直接有效求解。In each step of PI:1Policy evaluation involves solving a set of linear equations 2.Policy improvement: straightforward

Each step of policy iteration is guaranteed to strictly improve the policy at some state when improvement is possible./Converge to optimal policy./Gives exact value of optimal policy.

Start with a randomly chosen initial policy π_0

Iterate until no change in utilities:

1. Policy evaluation: given a policy π_i, calculate the utility $U_i(s)$ of every state s using policy π_i
2. Policy improvement: calculate the new policy π_{i+1} using one-step look-ahead based on $U_i(s)$ ie.

$$\pi_{i+1}(s) = \arg\max_a \sum T(s, a, s') U_i(s')$$

Policy improvement is straightforward

Policy evaluation requires a simpler version of the Bellman equation

Compute $U_i(s)$ for every state s using π_i:

$$U_i(s) = R(s) + \gamma \sum_{s'} T(s, \pi_i(s), s') U_i(s')$$

Comparison: Which would you prefer, policy or value iteration? 1.have lots of actions in each state: policy iteration 2.have a pretty good policy to start with: policy iteration 3.have few actions in each state: value iteration